A reliable batch pipeline for Spotify × Grammy data

Two sources · two quality gates · one trusted Data Warehouse

Michael Macowli Cardona Rodriguez · ETL (G01) · Workshop-2

> Reliable means failures are anticipated, controlled and observable.

2
sources: CSV + PostgreSQL

18
quality rules in 3 gates

113,549
rows loaded = rows prepared

5/5
reliability tests passed

Three questions that need both sources

Spotify knows how music sounds and how popular it is. The Grammys know what was awarded.

RQ-01

Do Grammy winners sound different, and are they more popular?

KPI-01 · audio profile + popularity uplift

RQ-02

In which genres does a Grammy matter most for popularity?

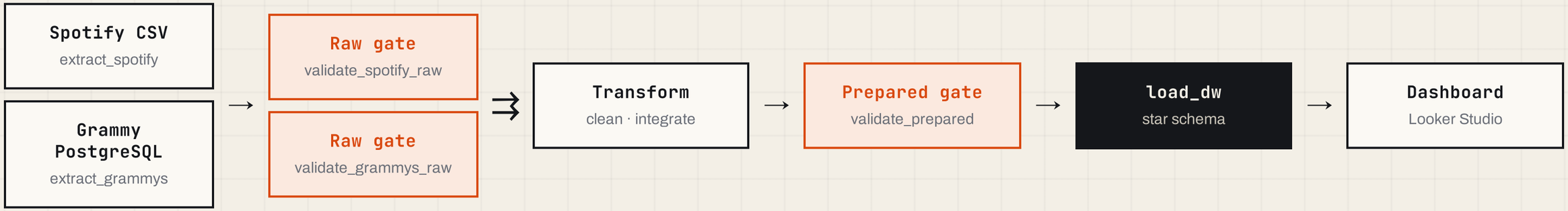
KPI-02 · popularity uplift per genre

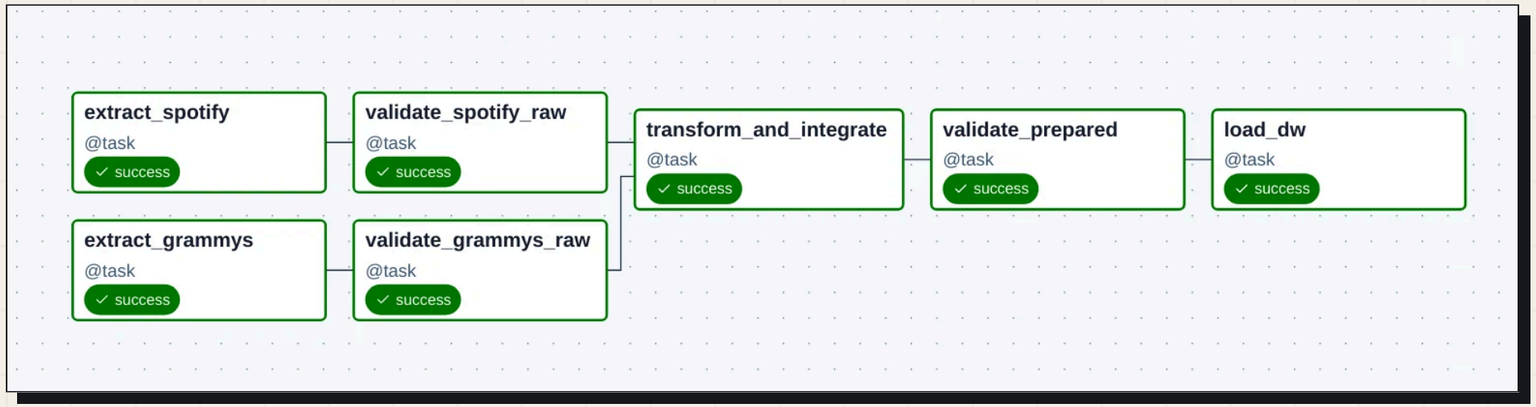
RQ-03

How have explicit content and collaborations evolved among winners?

KPI-03 · % explicit and % collaborations per decade

Two observable branches, two gates, one gated load





Airflow 3.1.8 · `from airflow.sdk import dag, task`

Transform waits for both gates. Load waits for the prepared gate.

XCom carries paths, not data. Each run keeps its own files and evidence.

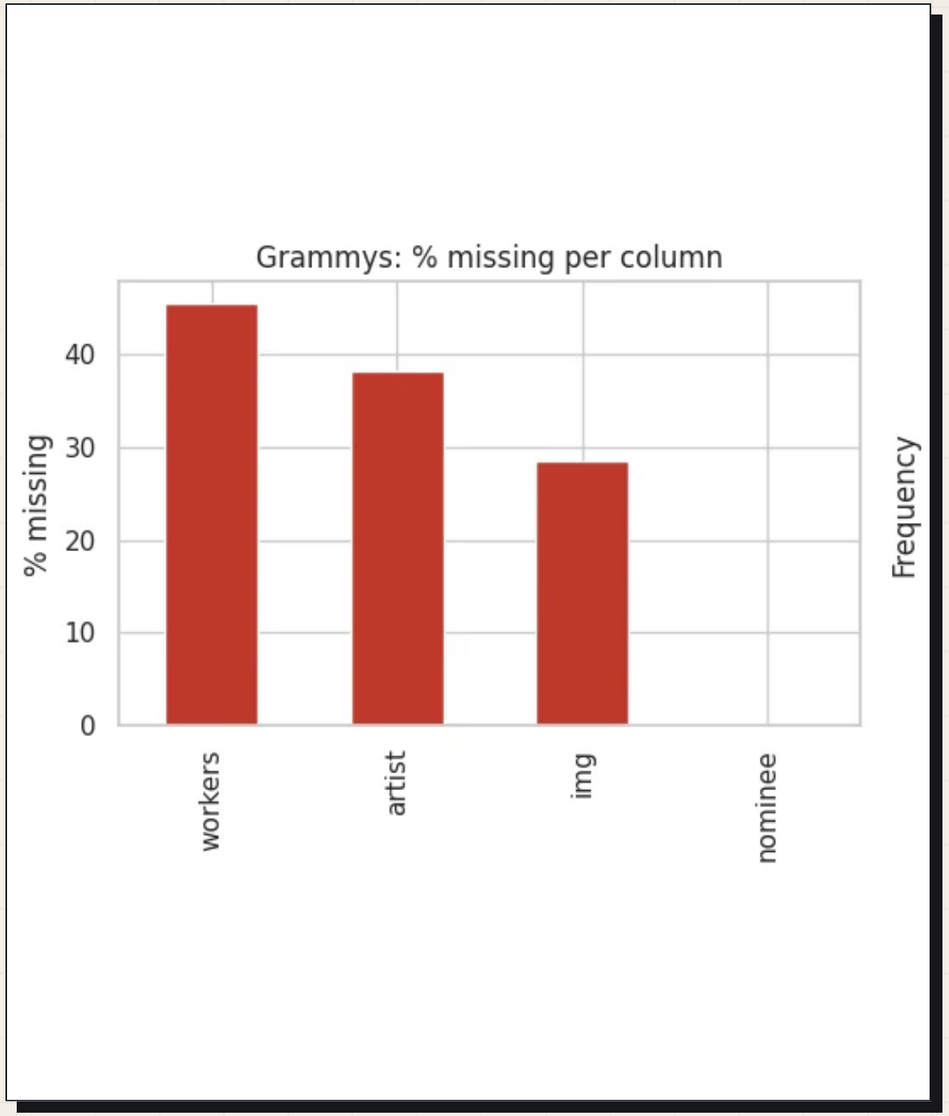
Profiling first: what the data really looks like

450
duplicated track × genre rows

1
corrupted row (0 ms, no title)

38%
Grammy awards without artist

100%
of Grammy rows are winners



18 rules, each with a threshold and a consequence

9

Spotify raw gate
schema · IDs · duration · ranges · duplicates

4

Grammy raw gate
schema · year 1958–2026 · nominee · volume

19

Prepared gate
schema · ranges · unique grain · integration · match rate

CRITICAL

Stop. Downstream blocked, DW untouched.

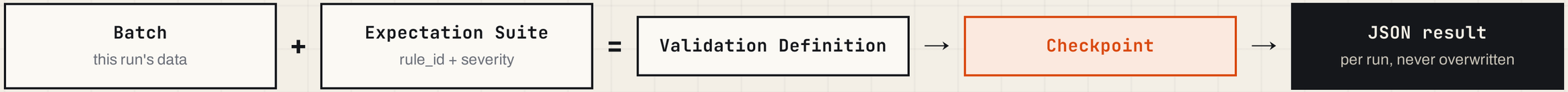
WARNING

Continue,
logged and recorded.

INFORMATIONAL

Record
as a trend signal.

Every gate runs a GX Checkpoint




```
// evidence of the controlled failure (Test B)
"gate_decision": "STOP",
"rule_id": "DQ-RAW-GRM-01",  "severity": "Critical",
"expectation_type": "expect_column_values_to_be_between",
"unexpected_count": 1,
"partial_unexpected_list": [1800]
```

32 / 32

expectations pass on the good batch

STOP

names the rule and the bad value

Every row is accounted for



Key

normalized title + primary artist

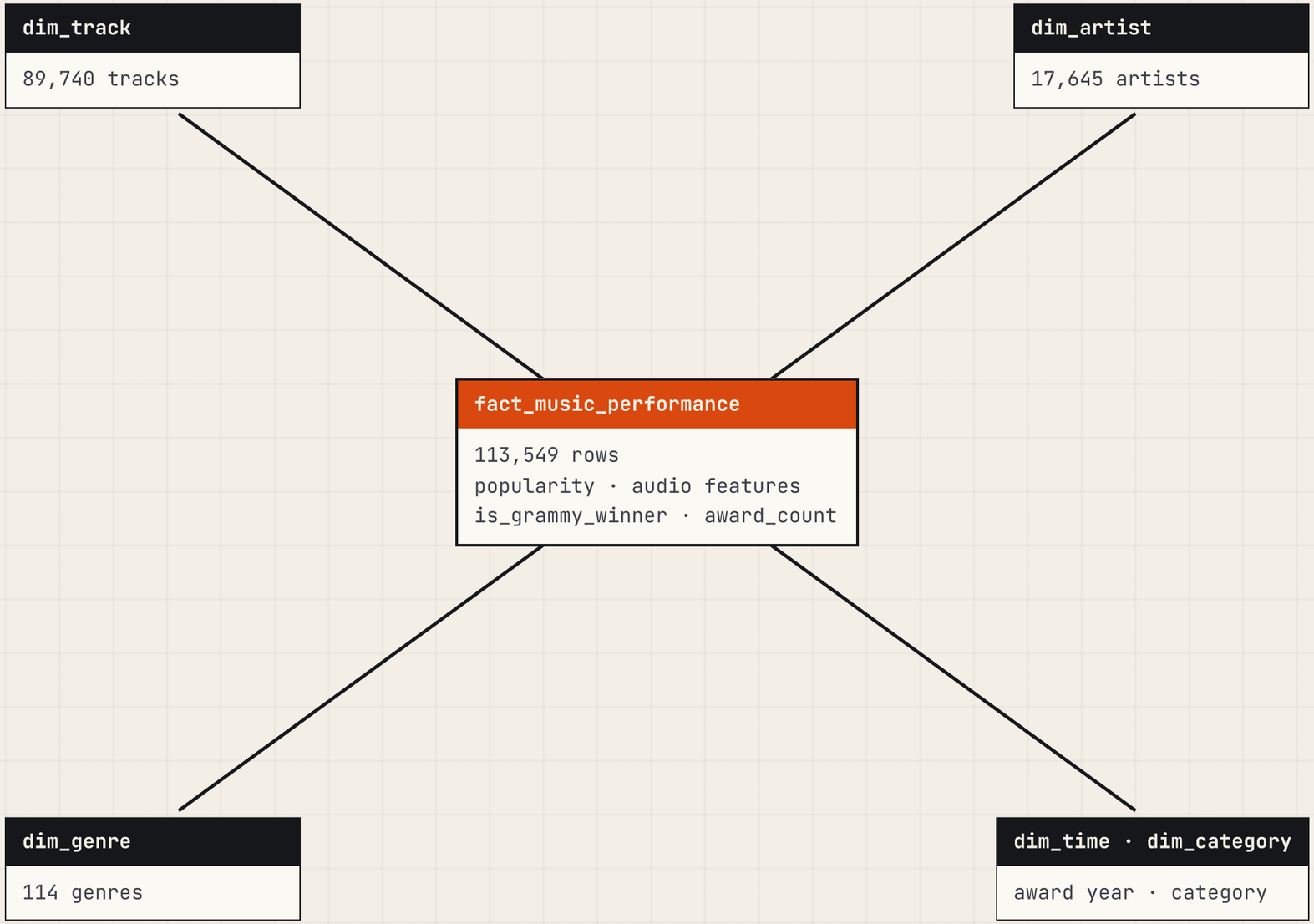
Many-to-one, enforced

a catalog row matches one award record at most

282

awarded tracks found in the catalog

A star schema built for the three questions



Grain

one row per track × genre × award year × category

Surrogate keys + unique business keys

KPI views on top: the dashboard reads them, never the raw tables

Retry only when another attempt can succeed

Transient → retry

database briefly unavailable · file not mounted · lock timeout

2

retries for extracts
15 s, exponential backoff

1

retry for load_dw
one atomic transaction

Deterministic → stop

Critical rule violated · broken schema · code error

0

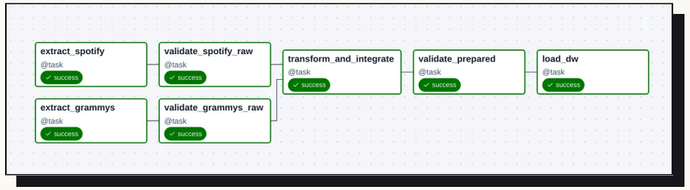
retries for the 3 gates and the transformation:
the same batch fails the same way

Live demo

1

Run the pipeline

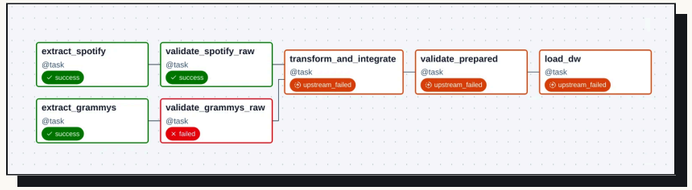
7 green tasks, 113,549 rows loaded



2

Break the source

a year 1800 stops the raw gate



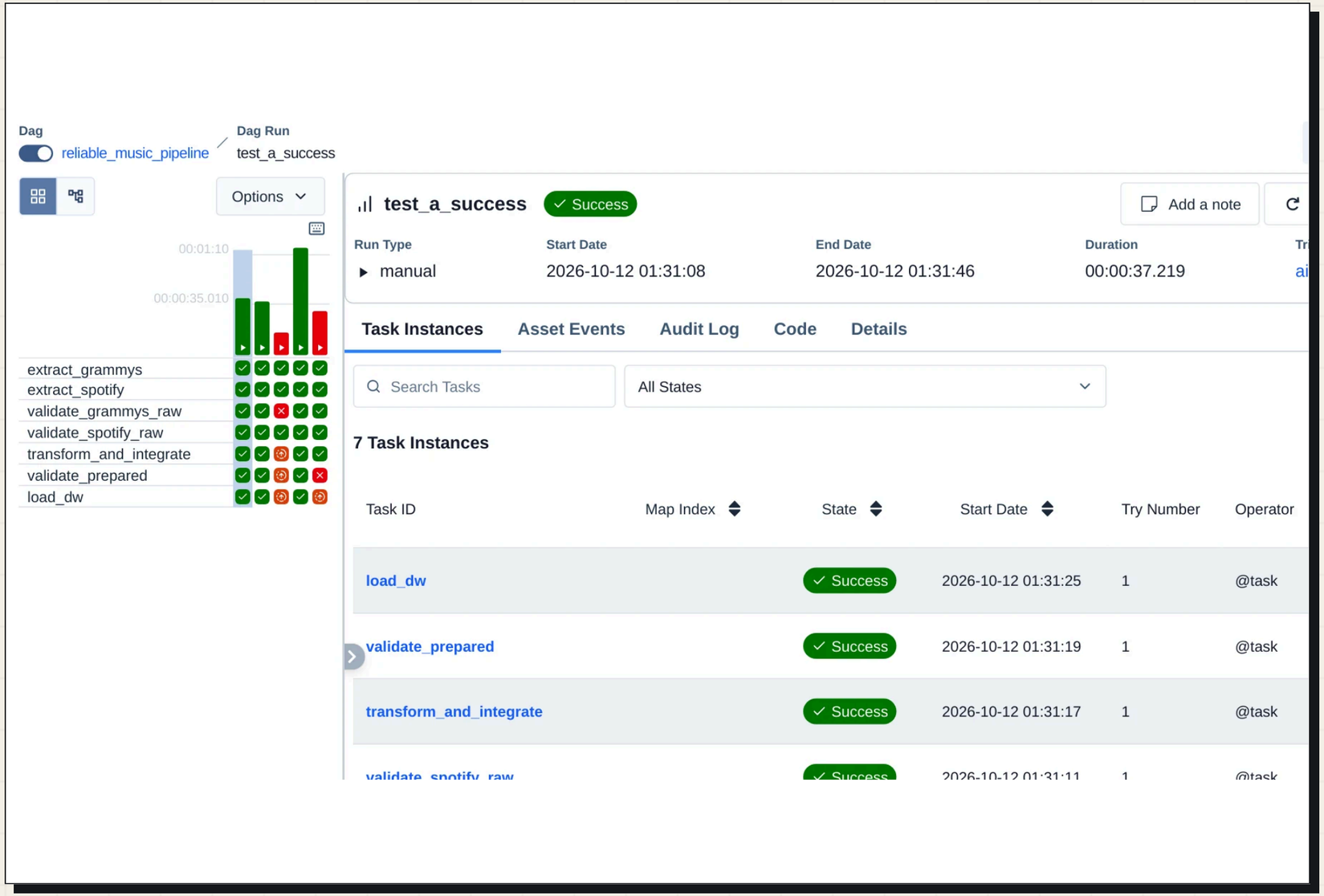
3

Fix and rerun

same rows, same totals, no duplicates

```
[2026-10-12 01:32:20] INFO - DW schema verified/applied from /opt/airflow/sql/dw_sch
[2026-10-12 01:32:20] INFO - DW state BEFORE load: {'fact_rows': 113549, 'fact_winne
[2026-10-12 01:32:35] INFO - ============================================================
[2026-10-12 01:32:35] INFO - DATA WAREHOUSE LOAD RECONCILIATION SUMMARY
[2026-10-12 01:32:35] INFO -  batch_id: test_c_safe_rerun
[2026-10-12 01:32:35] INFO -  prepared rows: 113,549 | inserted: 0 | updated: 113,54
[2026-10-12 01:32:35] INFO -  fact rows before -> after: 113,549 -> 113,549
[2026-10-12 01:32:35] INFO -  sum(popularity) before -> after: 3,783,956 -> 3,783,95
[2026-10-12 01:32:35] INFO -  winner rows: 475 | tracks: 89,740 | artists: 17,645
[2026-10-12 01:32:35] INFO - ============================================================
[2026-10-12 01:32:35] INFO - Load evidence saved to: /opt/airflow/docs/evidence/runs/
[2026-10-12 01:32:35] INFO - Done. Returned value was: {'batch_id': 'test_c_safe_rer
[2026-10-12 01:32:35] INFO - Pushing xcom ti=RuntimeTaskInstance(id=UUID('01a12dbc-f
```

Test A · a clean run, end to end



7/7

tasks succeeded in 36 s

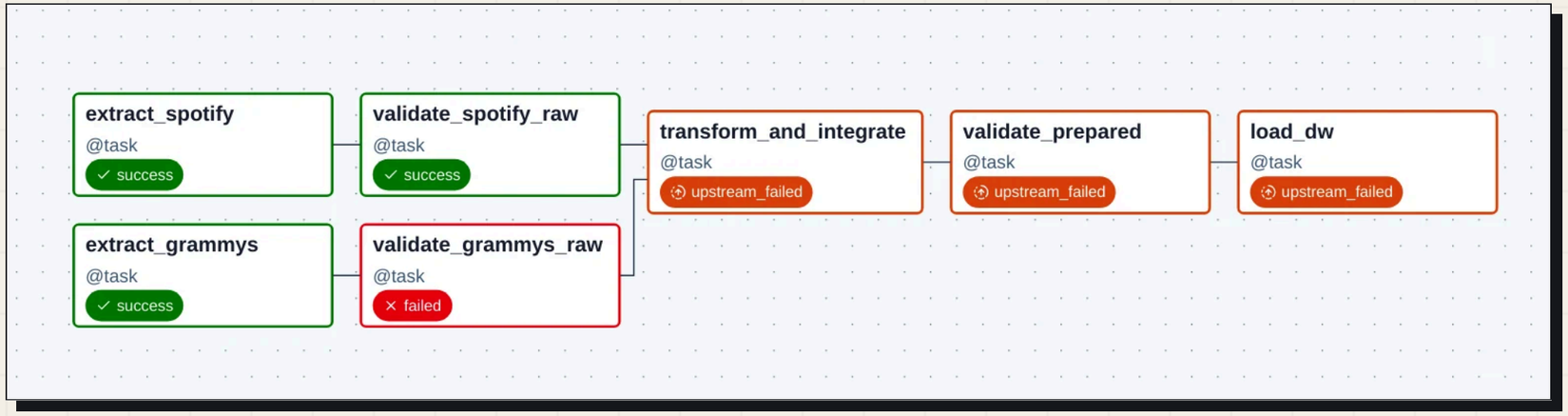
32/32

quality checks passed

113,549

rows inserted = rows prepared

Test B · one bad year stops everything before the load




```
223  [2026-10-12 01:33:00] INFO - [grammys_raw] Validation evidence saved to: /opt/airflow/docs/ev
224  [2026-10-12 01:33:00] INFO - [grammys_raw] 3/4 expectations passed on 4,811 records -> gate de
225  [2026-10-12 01:33:00] ERROR - Critical quality gate failure in grammys_raw: DQ-RAW-GRM-01 (yea
226  [2026-10-12 01:33:00] ERROR - Task failed with exception
                                   ▼ CriticalDataQualityError: Critical quality gate failure in
                                   File "/home/airflow/.local/lib/python3.11/site-packages/airf
                                   File "/home/airflow/.local/lib/python3.11/site-packages/airf
```

1800

award year injected in the source DB
DQ-RAW-GRM-01 · Critical

3

tasks blocked: transform, prepared gate, load

DW unchanged

113,549 rows from the previous good run

Test C · run it twice, get the same warehouse

fact rows · before **113,549**	=	fact rows · after **113,549**
sum of popularity · before **3,783,956**	=	sum of popularity · after **3,783,956**

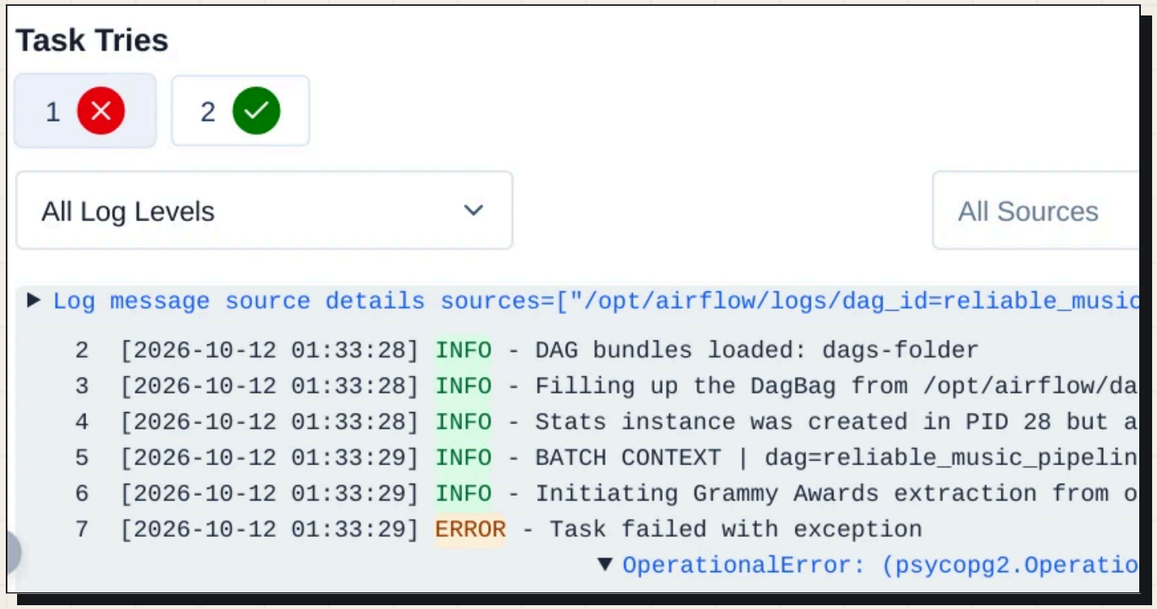
0 duplicates · only the audit stamp changes

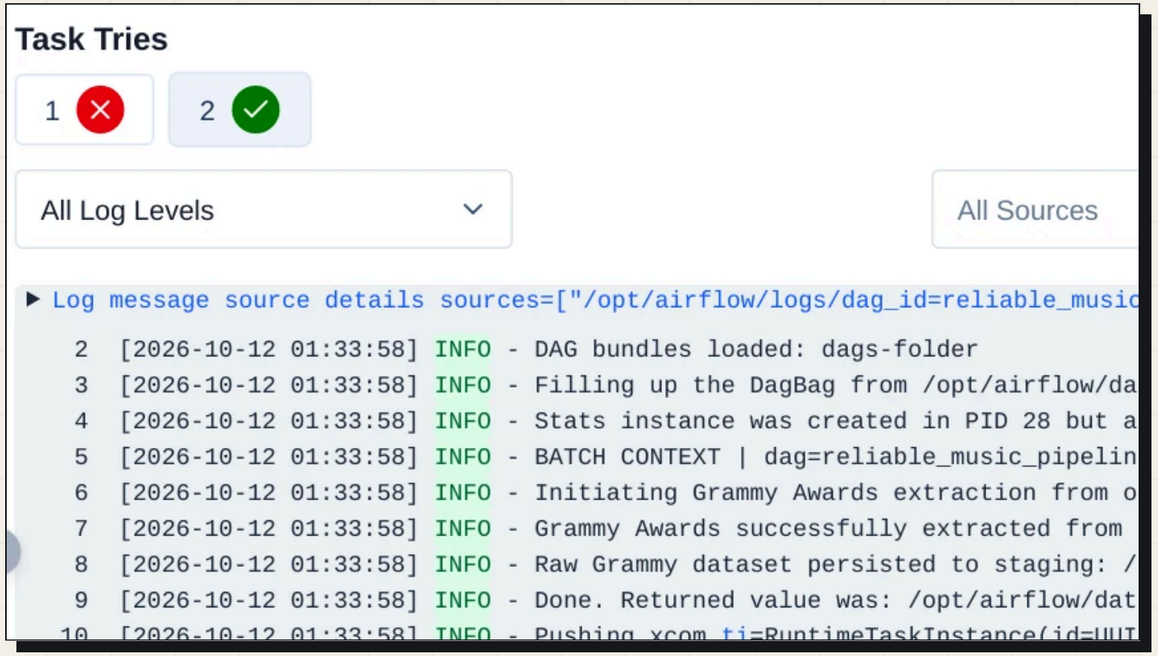
```
ON CONFLICT (track, genre, year, category)
DO UPDATE SET ...          -- 113,549 updated

DELETE rows NOT IN this run -- 0 purged

-- all in ONE transaction:
-- a failure rolls back to the
-- last good load
```

Test D · a short outage is retried and recovers



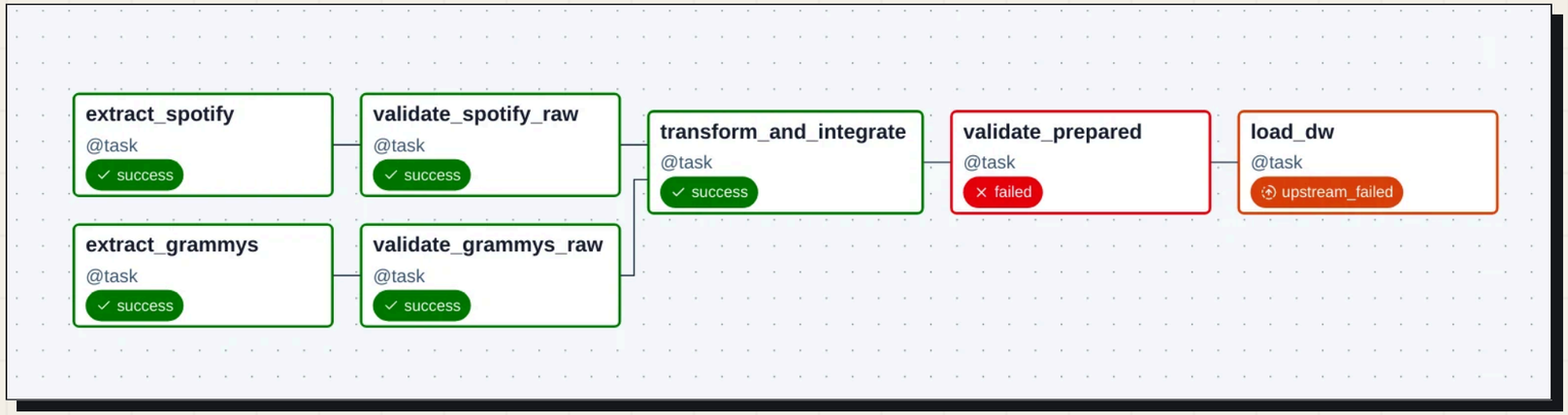


try 1
source DB refuses connections

wait
15 s with backoff

try 2
4,810 rows extracted, run succeeds

Test E · the source looks fine, the result does not



raw gates pass

artist credits changed format at the source

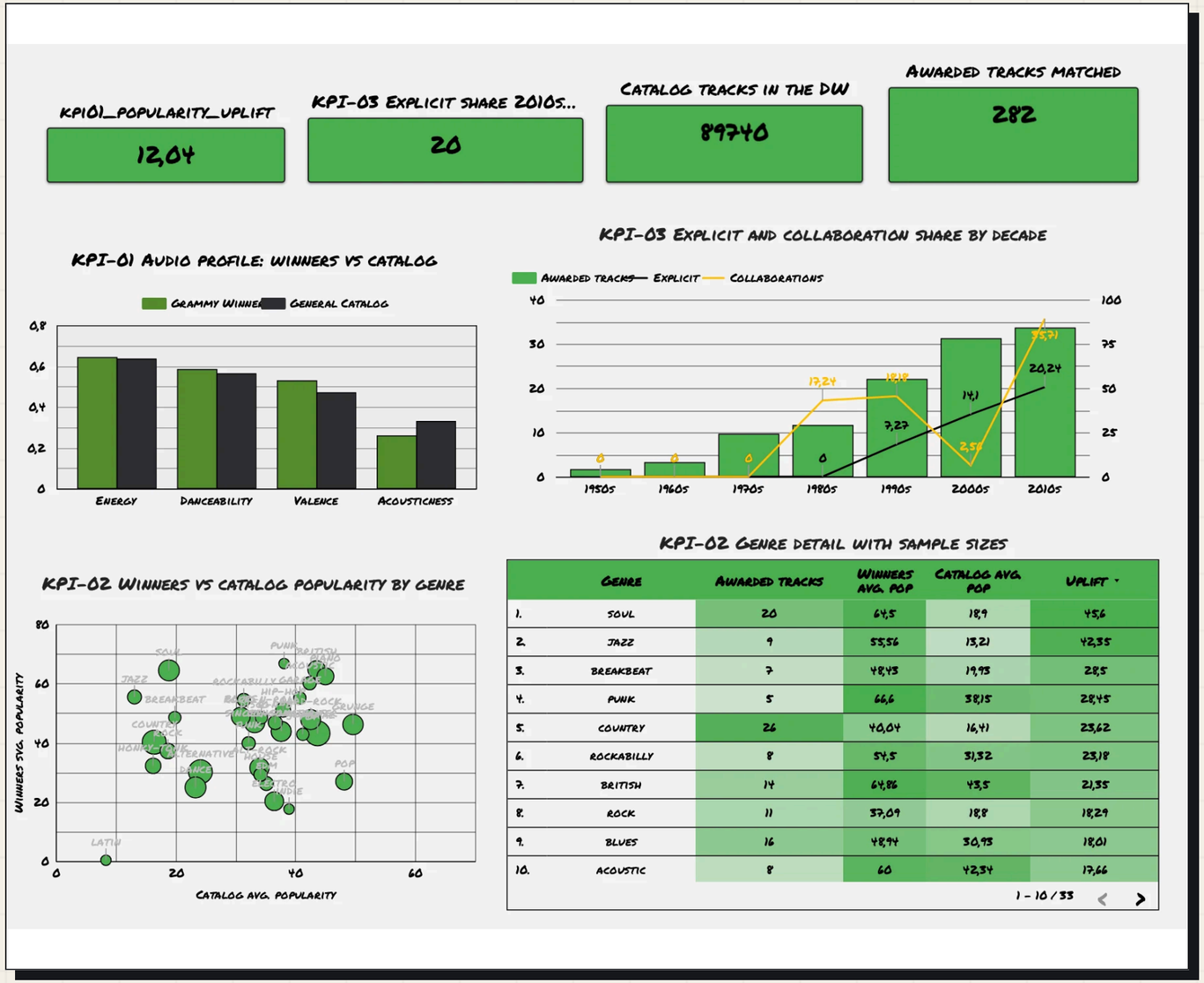
475 → 71

matched rows after the integration

load blocked

prepared gate: matched rows ≥ 0.1% · Critical

What the data says



+12.04

KPI-01 · popularity points of winners over the catalog

+45.6

KPI-02 · soul leads the uplift (n = 20)

20.2%

KPI-03 · explicit winners in the 2010s, up from 0%

Clone, set one file, run one command

```
cp .env.example .env
docker compose up -d --build

bash scripts/run_reliability_tests.sh    # Tests A–E
docker compose exec airflow \
  python /opt/airflow/scripts/publish_dw.py  # dashboard
```

Rebuilt from zero

volume deleted, everything came back with no manual steps

Evidence for every claim

JSON per run, logs, screenshots, evidence register

No secrets in git

credentials only in `.env`

Honest limits

Winners only
no nominee data in the source

Exact matching
4.1% of award keys match, measured every run

One snapshot
popularity shows association, not cause

Small samples
some genres have few winners; n is shown

> Reliable does not mean nothing fails.
> It means failures are anticipated, controlled and observable.

Questions

Why are validation tasks not retried?

Data-quality failures are deterministic: the same batch fails the same way (Tests B, E). Transient failures do retry (Test D).

What if load_dw fails halfway?

One transaction: everything rolls back and the previous snapshot stays. The retry converges because the load is idempotent.

How did you choose thresholds?

From profiling and the requirement they protect, not from what the current batch passes.

Why not pass DataFrames through XCom?

XCom lives in the Airflow metadata DB. Tasks pass file paths; data is kept per run for investigation.

Why a LEFT join, many-to-one?

Non-awarded tracks are the comparison group. Grammy rows are aggregated per key first, so a catalog row matches one record at most.

Why a cloud copy for the dashboard?

Looker Studio cannot reach localhost. The published copy is reconciled with the DW and read through a views-only role.